

Challenges in Scaling R-tree Spatial Search on Processing-In-Memory

Tasmia Jannat
Missouri S&T
Rolla, Missouri, USA
tjv7w@mst.edu

Michael Gowanlock
Northern Arizona University
Flagstaff, Arizona, USA
michael.gowanlock@nau.edu

Satish Puri
Missouri S&T
Rolla, Missouri, USA
satish.puri@mst.edu

Abstract

Spatial query processing is important in scientific, geospatial, and data-intensive applications. R-trees are widely used to index spatial objects, but their query-dependent traversal creates irregular work across different regions. This poster studies the challenges of scaling R-tree spatial search on a commercial Processing-in-Memory (PIM) system. Although PIM reduces CPU to memory data movement by executing search near memory, it does not remove full-pipeline overheads: the host still manages data placement, query batching, kernel launches, result retrieval, and aggregation. Our results show strong DPU-side search acceleration, with PIM kernel speedup ranging from about 20 $\times$ to 73 $\times$, but end-to-end speedup is lower, ranging from 0.87 $\times$ to 11.29 $\times$. The runtime breakdown shows that CPU-side aggregation can dominate output-heavy workloads; on the Buildings dataset, aggregation accounts for 62.9% of total time, while DPU kernel time is only 4.4%. DPU-count scaling shows that more DPUs speed up the kernel, but end-to-end gains saturate due to full-pipeline overheads. We also observe a workload imbalance across the DPUs, with the ratio of maximum to mean hits reaching 29.1 $\times$ on Lakes. These findings motivate parallel host-side aggregation, efficient result handling, and query-aware DPU assignment for scalable PIM-based spatial search.

Keywords

Processing-in-Memory, R-tree, spatial query processing, UPMEM, load balancing, communication overhead

ACM Reference Format:

Tasmia Jannat, Michael Gowanlock, and Satish Puri. 2026. Challenges in Scaling R-tree Spatial Search on Processing-In-Memory. In . ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Modern spatial workloads often search millions of rectangles, polygons, or multidimensional objects using range queries. R-trees are widely used because they organize objects with minimum bounding rectangles (MBRs), which help prune large parts of the search space. However, R-tree search is hard to scale because traversal is data-dependent: some queries visit only a few nodes, while others examine many branches and leaf rectangles. These irregular accesses make the search memory-intensive and cause significant data movement between memory and the host CPU.

Processing-in-Memory is promising for such memory-intensive workloads because it places lightweight compute units near memory, reducing repeated movement of R-tree nodes and rectangles from memory to the CPU cache for overlap checking. We use the commercial UPMEM PIM system, where each PIM chip contains 8 DRAM Processing Units (DPUs), and each DIMM contains 16

PIM chips, providing 128 DPUs and 8 GB of memory per DIMM. A 20-DIMM server can provide up to 2,560 DPUs and 160 GB of memory, enabling large-scale near-memory parallelism.

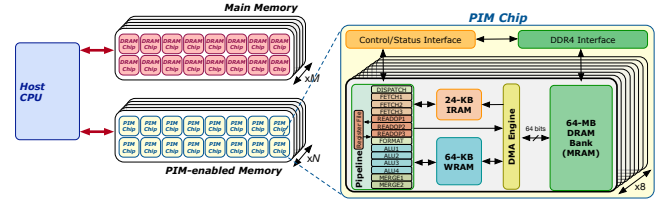


Figure 1: Overview of the UPMEM Processing-in-Memory architecture. Reproduced from Gómez-Luna et al. [2].

This poster studies R-tree spatial search on UPMEM PIM, building on our Broadcast PIM R-tree pipeline [3]. The CPU builds and partitions the index, while DPUs process batched queries in parallel. We focus on the scaling challenges of mapping irregular tree search to many DPUs. Our main observation is that DPU-side search can be fast, but end-to-end performance is limited by communication, result movement, and load imbalance.

2 Why Scaling R-tree Search on PIM is Difficult

Scaling R-tree search on PIM is difficult because the workload is irregular, skew-sensitive, and communication-dependent.

Irregular tree traversal. R-tree search depends on MBR overlap. Some queries prune most branches early, while others visit many nodes and leaf rectangles. Even if each DPU receives a similar amount of data, the actual search work may differ.

Spatial skew and query-induced hot regions. Spatial data and queries are often skewed. A DPU-owned tree partition can become hot if many queries overlap that region, even when its data size is similar to other partitions. Thus, equal leaf partitioning does not guarantee equal DPU work.

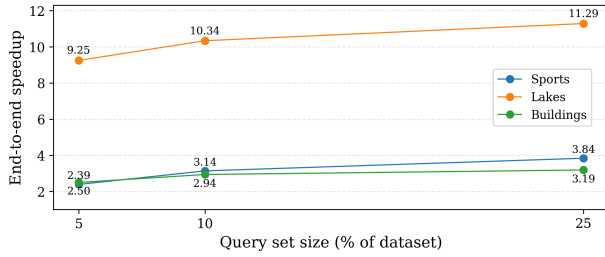
Full-pipeline overheads. PIM reduces data movement during local search, but it does not remove host-side orchestration. The host must still transfer inputs, retrieve results, and aggregate outputs. As DPU-side search becomes faster, these non-kernel stages can dominate end-to-end runtime, so performance must be evaluated beyond kernel time alone.

3 Results and Observations

We evaluate R-tree spatial search on Sports (1.7M rectangles), Lakes (8.4M rectangles), and Buildings (14.3M rectangles), using query sets of approximately 5%, 10%, and 25% of each dataset. All PIM runs use 2,540 DPUs and 11 tasklets per DPU. We report CPU

Table 1: CPU search time, DPU kernel time, total DPU time, and speedup across datasets.

Dataset	Query (% of Dataset)	CPU (s)	Kernel (s)	Total (s)	Kernel speedup (×)	End-to-end speedup (×)
Sports (1.7M)	5	4.68	0.23	1.96	20.51	2.39
	10	9.36	0.46	2.98	20.43	3.14
	25	23.60	1.14	6.14	20.67	3.84
Lakes (8.4M)	5	63.92	1.50	6.91	42.74	9.25
	10	126.07	2.98	12.20	42.24	10.34
	25	316.25	7.50	28.01	42.19	11.29
Buildings (14.3M)	5	22.96	0.32	9.18	72.54	2.50
	10	46.11	0.63	15.70	72.85	2.94
	25	114.65	1.58	35.89	72.59	3.19

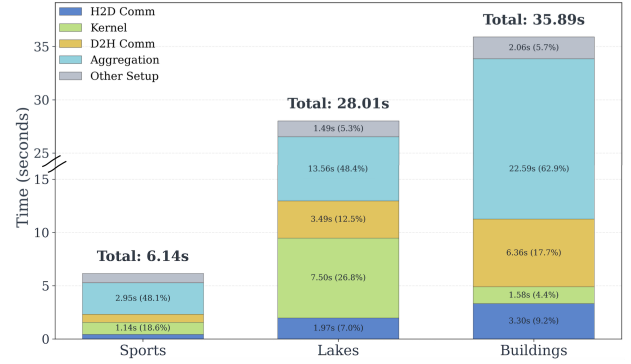
**Figure 2: End-to-end speedup improves with larger query sets, with dataset-dependent gains.****Table 2: Per-DPU load imbalance for the 25% query workload.**

Dataset [1]	Cycle max/mean	Hit max/mean
Sports	8.12	4.58
Lakes	4.88	29.12
Buildings	1.38	2.08

search time, DPU kernel time, and total DPU time, where total DPU time includes kernel execution, host–DPU communication, result movement, and host-side processing.

Table 1 shows that PIM strongly accelerates the DPU-side search kernel: about 20× on Sports, 40–42× on Lakes, and 72–73× on Buildings. However, kernel speedup does not directly translate to end-to-end speedup. For example, on Buildings with 25% queries, the kernel speedup is 72.59×, but the end-to-end speedup is only 3.19× because total DPU time includes communication, result movement, and host-side processing. Figure 2 shows that end-to-end speedup generally improves with larger query sets, suggesting better amortization of host–DPU overheads. However, Figure 3 shows that the dominant non-kernel cost is CPU-side aggregation. For Buildings, the DPU kernel takes only 1.58 s, while aggregation takes 22.59 s (62.9% of total time). This occurs because host aggregation is sequential and Buildings produces more result data.

Finally, Table 2 shows that irregular search creates dataset dependent DPU imbalance. Cycle max/mean compares the most-loaded DPU’s cycle count with the average DPU, while hit max/mean measures output imbalance across DPU-owned tree partitions. Sports has high cycle imbalance (8.12×), Lakes has extreme hit imbalance (29.12×), and Buildings is much more balanced. These results

**Figure 3: Runtime breakdown for the 25% query workload showing CPU-side aggregation dominates.**

show that equal leaf partitioning does not necessarily imply equal work, motivating efficient result handling, parallel aggregation, and query-aware workload balancing.

Conclusion

This poster presents lessons from mapping R-tree spatial query processing to a commercial PIM system. The results show three main observations. First, PIM can effectively accelerate the DPU-side R-tree search kernel. Second, end-to-end performance depends on full-pipeline overheads such as host–DPU communication, result movement, and CPU-side aggregation. Third, R-tree search remains irregular even with balanced leaf partitions, because query-dependent traversal can create hot DPU-owned tree regions.

Overall, scaling irregular spatial search on PIM is limited not only by DPU-side search time, but also by full-system effects. In the current implementation, result aggregation is sequential on the host CPU and can dominate output-heavy workloads. As ongoing work, we plan to parallelize host-side aggregation, explore DPU-side partial aggregation, and replicate hot regions across DPUs to spread query load.

Acknowledgments

This material is based on work supported by the National Science Foundation under Grants No. 2344578, No. 2402987 and No. 2402988.

References

- [1] Saheli Ghosh, Tin Vu, Mehrad Amin Eskandari, and Ahmed Eldawy. 2019. UCR-STAR: The UCR spatio-temporal active repository. *SIGSPATIAL Special 11*, 2 (2019), 34–40.
- [2] Juan Gómez-Luna, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo F Oliveira, and Onur Mutlu. 2022. Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system. *IEEE Access* 10 (2022), 52565–52608.
- [3] Tasmia Jannat, Michael Gowanlock, and Satish Puri. 2026. Parallel R-tree-based Spatial Query Processing on a Commercial Processing-in-Memory System. *arXiv preprint arXiv:2604.14445* (2026).